

Witness-CL: Learning Latent Continuations without Trusting the Proposer

Samuel Mausberg

AI-assisted research draft; author review and independent replication pending

Abstract

A deployed learner needs reusable predictive state, not just a larger memory. We extend an evidence-preserving continual-learning reference from supplied world models and observable state to an unknown finite deterministic transducer. The learner consumes only executed action–output traces under a declared state bound, retains an exact set of partial models, trains a small proposal network online, and checks immutable candidate programs against the previous incumbent in every compatible completion. Paired symbolic execution closes identical future programs only after their hidden states also coincide. Informative exploration is charged a non-refundable worst-case deficit before execution. We give conditional written proofs of exact cover, reset-value retention independent of proposal quality, and a completed-prefix exploration bound. In 38,400 held-out synthetic episodes, the budgeted learner matches a strong full-history planner’s late return but trails its average return by 0.0371 reward per episode. Informative probes reduce the worst observed prefix deficit from 40 to 4 relative to an unguided ablation. A shared-suffix optimization improves selected CPU checks but slows unrelated programs. The release contains 355 passing Python tests, 400 new and 632 legacy passing C++ cases, and 50 uncompiled Lean theorem attempts. It establishes neither a language-agent benchmark win nor general no-forgetting under misspecification, hidden drift, or fixed total storage.

1 Problem and the actual advance

CL-Bench evaluates whether frontier language agents improve across episodes that share learnable latent structure. Its tested raw-history in-context learning (ICL) systems outperform dedicated memory systems, with failures including immediate-observation overfitting and weak reuse across instances [1]. AgentCL uses controlled compositional, naive, and held-out streams to distinguish reuse from mere persistence; irrelevant memory can degrade performance [2]. These results motivate a more exact question: what part of experience should a learner treat as state, and what evidence warrants letting that state change its future behavior?

The open target is a cost-effective, deployed learner

that improves across useful stateful tasks from its own episodes without an offline retraining phase and without losing valuable prior behavior. Merely preserving weights, transcripts, or a skill file does not imply preserving downstream competence. Conversely, not every episode can be required to improve if useful information sometimes requires a costly experiment. A meaningful solution must separate retained competence from exploration loss and account for both.

Starting point. The uploaded Witness-CL v0.3 reference provides a finite list of possible worlds, observable complete state, continuation-aware improvement, incumbent retention, and prepaid exploration. Its synthetic late return matches rather than beats a strong full-history learner. Separate noisy-evidence and frozen-neural-module branches do not solve representation acquisition. We preserve that Git history and archive its paper and results rather than relabel them as new experiments.

Version 4. We replace the supplied world list and observed latent state with a learned set of hidden transducers consistent with executed traces. Only a structural class bound, reset semantics, finite alphabets, and public reward functions are supplied. The same loop now contains online neural proposal updates, exact model-set conditioning, immutable observation-contingent programs, paired continuation verification, and informative prepaid probes. No hidden world table, state, or regime label is given to the learner.

The advance is deliberately restricted. Discovering compatible latent models inside a correctly declared finite class is not discovering a sufficient abstraction of arbitrary language or computer-use traces. Our proposed high-impact direction is to learn affordable continuation contracts: compact predictive descriptions sufficient to justify useful behavioral changes. The reference makes this direction executable and falsifiable. Its current empirical result is not benchmark superiority.

2 Prior work constrains the claim

Memory and executable experience. ACE develops evolving playbooks; ALMA meta-learns memory designs; EvoLib builds and consolidates reusable knowledge

abstractions [3, 6, 12]. These are substantive adaptation mechanisms, not simply retrieval stores. ALMA’s outer search also makes it important to distinguish prior design cost from strictly online adaptation. None of these comparisons has been reproduced here.

EvoTest evolves system configurations between episodes, while TTHe evolves the executable harness and commits changes using execution-derived proxy signals [4, 5]. A September 4, 2026 study constructs persistent versioned skills for computer-use agents, freezes the library during each iteration, and reports benefits over a configuration-matched empty-library control along with revision failures [13]. Versioned libraries and frozen execution snapshots are therefore not new contributions of Witness-CL. The unresolved distinction is when evidence supports a claim about later behavior, rather than only a favorable proxy or recent task.

Latent state and conservative learning. Predictive state representations already define state through predictions of future observable experiments [10]. Dynamic shielding learns an approximate automaton from black-box interaction and filters potentially unsafe exploration [11]. DeepSPI already combines online safe policy improvement, learned world models, and representations, with conditional theory and ALE-57 evaluation [8]. These are direct precedents, not peripheral related work. Our exact deterministic class is narrower than those applications.

Conservative exploration methods such as StepMix address explicit baseline performance constraints [9]. The prepaid debit argument here is not the first conservative-exploration theorem. Progressive networks already isolate previous learned components [7]; neither freezing old functions nor changing parameters only online is novel by itself. We claim an implemented combination with explicit failure tests, not established novelty of the combination or a new universal learning principle.

3 A precise latent-learning contract

Let A and O be known finite action and output alphabets. The unknown world m^* is a stationary deterministic transducer with at most K states, embedded in $S = \{0, \dots, K-1\}$. A complete table specifies

$$T_m : S \times A \longrightarrow S \times O. \quad (1)$$

Every episode begins at the same actual reset state 0. Hidden state labels and the table are never observed. The learner receives the executed trace $\tau = ((a_0, o_0), \dots, (a_{H-1}, o_{H-1}))$ at finite horizon H . There is no stochastic feedback in the integrated experiment.

A public objective g specifies a known bounded integer reward $r_g(o)$. A policy p is a complete immutable tree:

$p_t : O^t \rightarrow A$. The reset return is

$$V_m(p, g) = \sum_{t=0}^{H-1} r_g(o_t), \quad a_t = p_t(o_{<t}). \quad (2)$$

A protected incumbent $b_{n,g}$ is stored for each declared objective. The objective identifies the requested utility, not a hidden dynamics regime. The common initial anchor is $b_{0,g}$.

Let $\mathcal{D}_n$ contain all actually executed traces before episode n . Define

$$\mathcal{C}_n = \{m \in \mathcal{M}_K : \forall \tau \in \mathcal{D}_n, m \models \tau\}. \quad (3)$$

Here $m \models \tau$ means that the table realizes the observed trace. The model class has $(K|O|)^{K|A|}$ labelled complete tables. Supplying K is a substantial prior. Data consistency cannot prove that $m^* \in \mathcal{M}_K$. Theorems below explicitly assume this coverage and unchanging reset semantics.

Four different claims. Preserving raw evidence is not the same as preserving a learned function. Preserving that function is not the same as preserving a routed incumbent’s reset value. Preserving incumbent value does not require every exploratory episode to attain that value. We prove the third property under this contract and separately bound the fourth property’s possible deficit. We do not prove that the mutable neural network itself avoids forgetting.

4 Learning an exact partial-model cover

A partial table c assigns some state–action edges and leaves the rest undefined. Its denotation is

$$[c] = \{m \in \mathcal{M}_K : c \subseteq T_m\}, \quad (4)$$

where inclusion means agreement on every assigned edge. A finite frontier $\mathcal{F}_n$ represents $\mathcal{C}_n = \bigcup_{c \in \mathcal{F}_n} [c]$. Initially the single all-undefined table represents the entire class. No complete list of worlds is passed to the learner.

To process one reset trace, pair every frontier table with state zero. For observed (a, o) , retain a defined edge only when its output is o . For an undefined edge, branch over all K successor states, assigning output o in every branch. Retain the branch’s hidden successor while processing the trace. Merge duplicate table–state pairs. At episode end discard the end state, since the next episode resets, and merge duplicate tables. Undefined edges are not filled with a likely default during inference.

Theorem 1 (Evidence-complete cover). *With exact feedback and no search truncation, the update above represents exactly $\mathcal{C}_n$. If $m^* \in \mathcal{M}_K$, every updated cover contains m^* .*

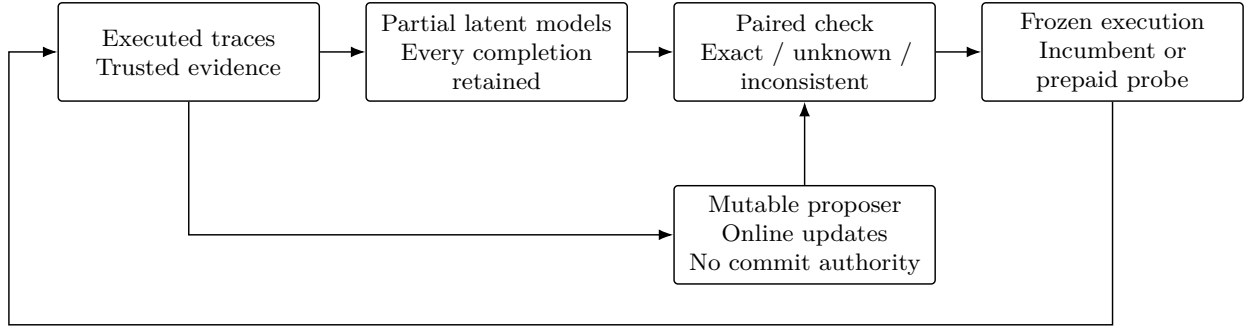


Figure 1: Implemented loop. Model rollouts do not become observed training labels. The proposer may forget internally; only checked immutable programs replace a protected incumbent. All guarantees are conditional on a correct finite class, stationary dynamics, declared rewards, and genuine resets.

Proof. Induct on observations within each reset trace. For any compatible complete table, a defined visited edge must have the observed output; an undefined edge has its true successor among the K branches. Thus no compatible table is lost. Conversely, every retained branch assigns the observed output to the visited edge and tracks its successor, so all its completions realize the processed prefix. Resetting the branch state between episodes is valid by assumption. Deduplication changes representation, not its union of completions. Induction over episodes gives equality; actual feedback gives truth membership. $\square$

This cover can overlap and can grow exponentially. The implementation caps the frontier and marks incomplete inference rather than silently retaining only a likely subset. Once coverage is unknown, new certificates are not issued. An empty exact class is reported as INCONSISTENT, never as vacuous success.

Changing capacity. Appending a new candidate program does not change $\mathcal{C}_n$. Increasing K does. The implementation replays all raw evidence under the new bound, starts a new contract era from the anchor, and does not inherit old certificates or the old budget claim. This is an explicit online recovery operation, not evidence that a fitted smaller model was sound before expansion.

5 Checking a pair rather than trusting a model

For a candidate p and current incumbent b , define

$$L_n(p, b, g) = \min_{m \in \mathcal{C}_n} [V_m(p, g) - V_m(b, g)], \quad (5)$$

$$U_n(p, b, g) = \max_{m \in \mathcal{C}_n} [V_m(p, g) - V_m(b, g)]. \quad (6)$$

Computing these extrema separately by materializing every complete world is unnecessary when many transitions cannot affect the comparison.

The checker symbolically executes both programs in the *same* partial table. It expands an undefined edge only when one execution reads it, branching over its $K|O|$ possible assignments. Both executions share each assignment. It accumulates their reward difference and retains a complete witness for the minimum. Unvisited slots remain universally quantified.

Closing a suffix. At the same time index, if both executions have the same hidden state and identical remaining observation-contingent programs, their remaining reward difference is zero for every completion. This follows by induction on future steps. Equal current observations alone do not justify closure: hidden states with the same emitted symbol can have different successors.

We compile remaining program trees into exact pair-local directed acyclic graph (DAG) IDs. The interning key is the complete tuple consisting of the current action and child IDs. Dictionary equality checks the whole tuple, so a hash collision cannot equate different programs. This is not a shapes-only cache or a learned equivalence prediction. DAG construction cost is included in timings.

Theorem 2 (Exact paired extrema). *For a nonempty exact cover, a completed paired search returns precisely (5) and (6). A returned lower-bound witness is a compatible complete table attaining the minimum.*

Proof. Expanding an unread edge partitions its completions into all possible assigned edges. A defined edge needs no split. Therefore every compatible world follows a search branch, and no branch introduces an incompatible visited edge. At the horizon the accumulated difference is constant over all unread completions. The same holds at an early closure because subsequent states, programs, observations, and rewards agree inductively. The minimum and maximum of these leaf differences are consequently the desired extrema. Any remaining undefined slots at a minimizing leaf may be completed arbitrarily, producing a compatible witness with the same difference. $\square$

A node cap yields UNKNOWN with no usable bound. It does not turn visited leaves into a confidence interval. For each partial, at most $u = \min(K|A|, 2H)$ distinct unresolved slots are read along a paired path, giving up to $(K|O|)^u$ assignments, in addition to horizon and frontier overhead. Full policy-tree construction itself costs $O(\sum_{t=0}^{H-1} |O|^t)$. Conditional sharing is not polynomial-time learning.

Corollary 3 (Maximality for the declared class). *For a fixed candidate and incumbent, $L_n \geq 0$ is necessary and sufficient for nonnegative return difference in every world of $\mathcal{C}_n$. If $L_n < 0$, its witness refutes uniform admission within this contract.*

Proof. Sufficiency is the definition of a minimum. Necessity follows because the finite nonempty class attains that minimum. Theorem 2 supplies a witness. $\square$

This does not make the overall learner optimal. A different experiment, candidate, objective, or weaker guarantee may yield higher return.

6 Online training and informative exploration

6.1 The proposer has no admission authority

The reference trains a small tanh network after every executed episode. Features combine the public objective with per-time action frequencies of a candidate program. With normalized observed return y_n , the loss is

$$\ell_n(\theta) = \frac{1}{2} (f_\theta(g_n, p_n) - y_n)^2. \quad (7)$$

One stochastic gradient update uses the selected program and its actual return. There are no evaluator-oracle labels, imaginary rollout labels, or offline optimization stages. A visit bonus encourages underexplored proposals. These features collapse distinct reactive trees and are intentionally weak; the primary library contains only open-loop words.

Every proposal is a typed immutable program. A candidate replaces the incumbent only after an exact check proves strictly positive L_n ; ties preserve the incumbent. An optional local-model driver accepts bounded JSON words or trees, not executable code or self-issued certificates. Its parser is tested, but no real LLM service was run. It is not a native benchmark adapter.

Theorem 4 (Proposer-independent retention). *Within a stationary realizable era, suppose every incumbent replacement is an immutable program with a completed check satisfying $L_n \geq 0$. Then for each declared objective g , the true reset values of its successive incumbents are nondecreasing, regardless of how proposals are generated or updated.*

Proof. Theorem 1 retains $m^* \in \mathcal{C}_n$. Substituting it into the universal bound gives $V_{m^*}(p, g) \geq V_{m^*}(b, g)$. Induct over admitted replacements. Proposal quality does not occur in the inequality, only in which candidate is checked. Frozen program semantics and unchanged objective/reset semantics are necessary for the substitution. $\square$

6.2 A non-refundable exploration ledger

For a non-admitted probe p_n , charge before execution

$$d_n = [-L_n(p_n, b_{n,g_n}, g_n)]_+. \quad (8)$$

Only a completed exact comparison can authorize a probe, and cumulative debits must remain at most B . Favorable realized outcomes do not refund debits. Incumbent execution and admitted improvements have zero debit.

Theorem 5 (Completed-prefix deficit). *Under Theorem 4’s assumptions, every completed episode prefix N satisfies*

$$\sum_{n=1}^N [V_{m^*}(b_{0,g_n}, g_n) - R_n] \leq \sum_{n=1}^N d_n \leq B. \quad (9)$$

Proof. Retention gives $V_{m^*}(b_{0,g_n}, g_n) \leq V_{m^*}(b_{n,g_n}, g_n)$. Exact debit validity bounds the latter by $R_n + d_n$ for each executed probe; the same relation holds with $d_n = 0$ for incumbent or admitted execution. Summation and the prepaid ledger yield the claim for every completed prefix. $\square$

Equation (9) allows individual losses. It is not an all-state performance guarantee, a guarantee under hidden dynamics changes, or a safety specification for irreversible actions. It applies separately within each contract era, not across unbounded capacity restarts.

6.3 Do not pay for a certainly uninformative probe

Our initial integrated controller could spend repeated debits on programs whose observable outcome was already fixed. We now enumerate a feasible probe’s set of distinct observable traces over $\mathcal{C}_n$. A probe is selected only when this set contains more than one trace and $U_n > 0$.

Proposition 6 (Strict evidence reduction). *If a deterministic probe has at least two possible observable traces in $\mathcal{C}_n$, every realized trace removes at least one currently compatible world.*

Proof. Two different possible traces cannot both equal the observed trace. A compatible world realizing the other trace is eliminated by conditioning. $\square$

Algorithm 1: One reset episode of the implemented learner

Input: Exact or flagged cover $\mathcal{F}_n$; public goal g ;
immutable incumbent b_g ; spent debit D ;
budget B

Rank immutable candidate programs using the online proposer;
Check each candidate against b_g by paired symbolic execution;
if *some completed check has $L > 0$* **then**
 Replace b_g by a candidate with maximal L , using proposer rank to break ties;
 Execute this frozen program with zero debit;
else
 Among exact checks with $U > 0$ and $D + [-L]_+ \leq B$, enumerate possible traces;
 if *a feasible program has more than one possible trace* **then**
 Choose smallest debit, then most traces, then largest U , then rank;
 Debit before executing the frozen program;
 else
 Execute the incumbent without debit;
Condition the cover on the actual action–output trace;
Apply one neural update using only its observed return;

This is not expected information gain. The controller orders feasible probes by smallest debit, most distinct observable traces, largest optimistic return difference, then proposer rank. Counts are not probabilities and do not count arbitrary latent-state labelings as information. At most $|\mathcal{C}_0| - 1$ strictly eliminating events are possible in a finite class, but this exponential bound does not guarantee sufficient budget, available distinguishing programs, or identification of an optimal policy.

7 Executed experiments, including failures

7.1 Protocol and comparators

Development uses seeds 0–19. An initial seven-arm run contains 6,720 episodes; the revised eight-arm development run contains 7,680. A local protocol record was written before generating 100 fresh seeds, 10000–10099. It is a local freeze, not external preregistration. The primary run comprises eight systems, 48 episodes per seed, and 38,400 episode rows.

Each hidden transducer has two states, two actions, and two outputs. The horizon is four, and all 16 binary action words form the candidate library. The learner does not receive the generated table. Public rewards alternate in three 16-episode blocks: $r_A = (0, 2)$, $r_B = (2, 0)$, then r_A again. The transition model never changes. This is public-objective recurrence, not hidden-regime

recurrence. Episode rewards lie in $[0, 8]$.

The strong full-history comparator enumerates all consistent models and selects the word with largest optimistic value, breaking ties by mean model value and then lexicographic order. It has no risk constraint. It is a symbolic planner, *not* raw-history LLM ICL. A last-output model is a weak observation-reactive diagnostic. Other arms are the fixed anchor, Witness at budgets 0, 16, and 64, a no-training ablation, and an unguided-probe ablation.

All systems receive only their own executed feedback and have equal environment horizons. Total CPU time is not matched. Evaluator-only word oracles do not train the agents. The primary metric is mean return over all episodes; late-A return uses the last eight. We additionally track incumbent regressions, risk spent, and maximum completed-prefix deficit against the fixed anchor.

Held-out limitations and audit. The 256-table class is tiny. Development contains 19 distinct tables and the 100 fresh seeds contain 81, with six tables shared between the two sets. Independent fresh seeds therefore do not establish unseen-world generalization. A secondary new-reward diagnostic in raw JSON preserves objective A’s output ordering and is not treated as credible new-domain transfer.

After the first held-out run, exact DAG IDs replaced repeated structural suffix checks and strict candidate registration was added. The controller’s selection rule was not tuned on held-out outcomes. The primary run was repeated with the final code. An included audit verifies identical actions/programs, rewards, objectives, deficits, spent risk, and frontier sizes across all 38,400 rows. Original freeze hashes and pre-backend results are retained; timings and search counters are allowed to differ.

7.2 Learning and retention results

Table 1 shows improvement over the fixed anchor, but not over the strong planner. Guided budgets 16 and 64 have mean return 6.3358 versus 6.3729. The paired-seed difference is -0.037083 reward per episode, with a 95% paired t interval of $[-0.055284, -0.018883]$. Average-return superiority is rejected in this experiment. Late-A return ties at 6.48. The planner also has a lower largest observed prefix deficit, 2 versus 4. The conditional bound is not evidence of an empirical risk–return advantage over this planner.

The informative-probe refinement is useful within this controller. Guided B64 exceeds unguided B64 by 0.056667 reward per episode, interval $[0.029805, 0.083528]$, and reduces the largest observed prefix deficit from 40 to 4. Training exceeds the no-training B64 ablation by only 0.014167, interval $[0.007394, 0.020940]$. These secondary comparisons are

System	Mean	Late A	Max. deficit
Fixed anchor	3.9267	3.7800	0
Full-history planner	6.3729	6.4800	2
Last-output model	6.0275	6.1450	52
Witness, $B = 0$	6.2354	6.4000	0
Witness, $B = 16$	6.3358	6.4800	4
Witness, $B = 64$	6.3358	6.4800	4
No neural training, $B = 64$	6.3217	6.4800	4
Unguided probes, $B = 64$	6.2792	6.4800	40

Table 1: Executed primary study: 100 fresh seeds, eight systems, 48 episodes, 38,400 rows. Mean and late-A values are rewards, not percentages. Maximum deficit is the worst completed-prefix anchor deficit across all seeds. The strong full-history planner is symbolic, not an LLM ICL baseline.

descriptive and not multiplicity-corrected confirmatory findings. The small neural effect argues against presenting this as a major new training algorithm.

There are zero recorded incumbent regressions under the realizable contract. Mean B16 risk spent is 2.62, versus the allowed 16; mean incumbent promotions are 1.4 per seed. The network receives 48 actual-return updates per seed. Its weight payload is 1,544 bytes, but this excludes features, visits, trees, evidence, frontiers, archives and Python overhead. Raw trace payload grows with episode count. No bounded-total-memory result follows.

7.3 Checker cost and robustness

A separate 60-query CPU diagnostic compares paired execution with and without suffix closure, using seven timing repetitions per query and identical extrema. Median query speedups are 8.41 for identical programs, 1.55 for shared suffixes, and 0.90 for unrelated programs. The identical-program case is a trivial short-circuit, not an end-to-end systems claim. Unrelated programs remain slower. Before DAG IDs, repeated subtree work erased the shared-suffix benefit; those negative measurements are preserved.

A 15-case scaling stress test varies state count and horizon with a 20,000-node cap. Four cases return UNKNOWN. This is correct abstention but unfavorable scaling, not a hidden success. The reference has no new GPU kernel, and its CPU microbenchmarks do not predict GH200 serving performance.

An additional 1,200-episode diagnostic overwrites proposer parameters with large arbitrary values between episodes. Incumbent and budget violations remain zero under the contract. This tests separation of proposal and admission, not adversarial safety of the whole Python process. An unannounced dynamics-change counterexample has old certified gain +2 and actual new-world difference −2. Feedback can detect some contradictions after execution; it cannot protect the first violation of an unobserved assumption.

8 Implementation and proof status

The release adds an exact partial-table learner, paired checker, online agent, strict proposal parser, three experiment drivers, independent C++ rollout reference, and tests. All 355 Python tests pass. Exact small-class tests compare model covers and paired extrema against exhaustive enumeration, including reactive policies, lower-bound witnesses, empty classes, search caps, capacity changes, and wrong-state-bound counterexamples. The neural gradient is checked by finite differences.

The new C++ reference passes 400 paired observation-tree cases and a bounded reward rejection under undefined-behavior sanitization. The preserved legacy suite passes 632 cases plus an overflow check. These tests validate complete numeric rollouts and boundary handling, not a formal refinement of the symbolic search. No native benchmark, actual LLM inference, or CUDA experiment was run.

Eleven new Lean declarations attempt abstract cover, refinement, evidence intersection, guarded update, arbitrary proposal, retained chains, informative elimination, observation aliasing, budget lifting, and shrinking-class preservation. Together with the prior files there are 50 declarations. The local `lake build` attempt failed because the compiler was unavailable; release retrieval attempts also failed. None is kernel-checked. The absence of `sorry` or `admit` is not proof verification. Moreover, even a successful future build would not establish that Python implements the assumed cover or that a native task satisfies the contract.

9 Limitations and decisive next experiments

Where the guarantee breaks. Consistency with observed traces does not validate K , stationarity, reset semantics, deterministic feedback, or output sufficiency. A point estimate or underspecified class can certify a

change that fails in a larger compatible world. Hidden drift can invalidate a previous certificate before its first counterexample arrives. Public objectives do not solve hidden task routing. The fixed reset-value theorem is weaker than v3’s supplied-model all-state claim and much weaker than broad capability retention.

There is also an unavoidable information–risk tradeoff within some classes. Two worlds can make the anchor observationally identical while a distinguishing action has opposite reward consequences. Zero-risk execution cannot both retain the anchor in the unfavorable world and guarantee learning the favorable one. The right target is therefore not an unconditional monotonicity slogan, but a useful, explicit and affordable tradeoff. Independent new information can also require growing total storage; our archive does not bypass that cost.

Native falsification. The next experiment must first acquire a valid abstraction from a real, resettable tool environment without using hidden tables or task answers. Then evaluate compatible CL-Bench domains and AgentCL streams with the same fixed backbone, permissions, generation settings and total compute budget. Include raw-history ICL, a strong memory/skill baseline, an evolving harness, point-estimate models, and removals of checking, training and informative probes. Count proposal sampling, gradient updates, checker work, retrieval, replay, resets, retries and environment interactions. An oracle-representation arm is diagnostic only.

A concrete proposed success criterion is at least five percentage points of native normalized learning gain over the strongest cost-matched baseline, positive lower paired 95% bound, and no protected old-task success loss larger than two percentage points. These are research targets, not achieved results. Reject the scalable hypothesis if no sufficient small abstraction survives held-out continuation tests, if over 90% of candidate checks return UNKNOWN on the selected native tasks, if apparent gains vanish at matched compute, or if cost exceeds the strongest baseline without the predeclared gain.

Resources and execution design. The current simulator requires CPU only. A native pilot can use one GH200-class accelerator for a fixed local backbone and

a multicore Docker host for task isolation. Measure 100 pilot episodes before assigning a larger GPU-hour budget; no throughput or model fit has been measured here. Retain full provenance and measure actual peak host/GPU memory rather than payload estimates.

A possible systems extension groups partial-transition work by time and unresolved slot in structure-of-arrays queues, with exact residual IDs and checked reductions. CPU parallelism should precede a GPU rewrite for tiny irregular checks. A proof cache must include immutable programs, objective, reset contract and evidence ancestry; enlarging the class invalidates reuse. These are unimplemented engineering hypotheses. A GPU cannot remove exponential model ambiguity, and an approximation cannot inherit exact guarantees without a separate soundness argument.

10 Conclusion

Witness-CL v0.4 learns compatible hidden-state models from its own executed traces and uses them to protect a behavioral incumbent independently of a mutable proposer. It supplies exact conditional reasoning, tested implementation boundaries, informative-exploration evidence and reproducible negative results. It does not yet supply the requested general stateful benchmark winner. The remaining research obligation is concrete: learn sufficient continuation abstractions cheaply enough that verified reuse beats full-history learning at equal cost, without hiding class misspecification or old-skill loss behind the word “memory.”

Reproducibility and disclosure

Source, tests, raw episode files, result generation, original Git history, failed Lean build evidence and paper sources are included in the local research repository. The main reproduction targets are `make test`, `make cpp-check`, `make latent-heldout`, `make latent-diagnostics`, and `make paper`. The optional model-service driver was not executed. This is an AI-assisted draft prepared for Samuel Mausberg; independent review, actual proof compilation and external replication are required before strengthening any claim.

References

- [1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, Joseph E. Gonzalez. Continual Learning Bench: Evaluating Frontier AI Systems in Real-World Stateful Environments. 2026. <https://arxiv.org/abs/2606.05661>.
- [2] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, Yu Su. AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents. 2026. <https://arxiv.org/abs/2606.02461>.
- [3] Qizheng Zhang, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. 2025. <https://arxiv.org/abs/2510.04618>.
- [4] Yufei He, et al. EvoTest: Evolutionary Test-Time Learning for Self-Improving Agentic Systems. 2025. <https://arxiv.org/abs/2510.13220>.
- [5] Jun Nie, et al. TTHE: Test-Time Harness Evolution. 2026. <https://arxiv.org/abs/2607.08124>.
- [6] Yiming Xiong, Shengran Hu, Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. 2026. <https://arxiv.org/abs/2602.07755>.
- [7] Andrei A. Rusu, et al. Progressive Neural Networks. 2016. <https://arxiv.org/abs/1606.04671>.
- [8] Florent Delgrange, Raphael Avalos, Willem Röpke. Deep SPI: Safe Policy Improvement via World Models. In *International Conference on Learning Representations*, 2026. <https://arxiv.org/abs/2510.12312>.
- [9] Donghao Li, Ruiquan Huang, Cong Shen, Jing Yang. Near-optimal Conservative Exploration in Reinforcement Learning under Episode-wise Constraints. In *International Conference on Machine Learning*, 2023. <https://arxiv.org/abs/2306.06265>.
- [10] Satinder Singh, Michael James, Matthew Rudary. Predictive State Representations: A New Theory for Modeling Dynamical Systems. In *Proceedings of the Twentieth Conference on Uncertainty in Artificial Intelligence*, 2004. <https://arxiv.org/abs/1207.4167>.
- [11] Masaki Waga, Ezequiel Castellano, Sasinee Pruekprasert, Stefan Klikovits, Toru Takisaka, Ichiro Hasuo. Dynamic Shielding for Reinforcement Learning in Black-Box Environments. In *Automated Technology for Verification and Analysis*, 2022. <https://arxiv.org/abs/2207.13446>.
- [12] Weijia Xu, Alessandro Sordani, Chandan Singh, Zelalem Gero, Michel Galley, Xingdi Yuan, Jianfeng Gao. Test-Time Learning with an Evolving Library. 2026. Version 2, July 14. <https://arxiv.org/abs/2605.14477>.
- [13] Longtao Hu, Xiao Liang, Linchao Zhu. From Interaction Traces to Persistent Skills: Online Evolution for Computer-Use Agents. 2026. Submitted September 4. <https://arxiv.org/abs/2609.04869>.